

Standalone FPGA-Based QAOA Emulator for Weighted-MaxCut on Embedded Devices — 해설·정리본

항목	내용
원문 제목	Standalone FPGA-Based QAOA Emulator for Weighted-MaxCut on Embedded Devices
저자	Seonghyun Choi, Kyeongwon Lee, Jae-Jin Lee, Woojoo Lee
게재지 / 연도	arXiv 프리프린트 arXiv:2502.11316v2 [cs.ET], 2025-03-27
DOI	지면에 표기 없음 (arXiv ID: 2502.11316v2)
분량	8쪽 (본문 약 6.5쪽 + 참고문헌 1.5쪽), 6개 절, 그림 6개, 표 3개, 수식 20개
원문 파일	../papers/3. QCE-FPGA 논문/Standalone FPGA-Based QAOA Emulator for.pdf

원문 PDF 파일명이 ... QAOA Emulator for.pdf 로 잘려 있는데, 1쪽에서 확인한 실제 제목은 위 표의 것이 전체입니다.

한눈에 보기

FPGA 하나만으로 완결되는(standalone) 양자컴퓨팅 에뮬레이터를 만들고, 그 위에서 QAOA로 Weighted-MaxCut 문제를 푸는 논문입니다. 핵심은 QAOA의 유니터리(unitary) 행렬을 “**대각행렬 × 아다마르(Hadamard) 행렬**” 형태로 분해해 $O(N^2)$ 짜리 행렬-벡터 곱을 $O(N)$ 으로 낮춘 것, 그리고 곱셈은 클럭당 1개만 하고 덧셈만 N 개 병렬로 돌리는 파이프라인으로 DSP 사용량을 큐비트 수와 **무관하게 고정**시킨 것입니다. 그 결과 Kintex-7 중급 FPGA에서 기존 방식이 3큐비트에서 합성 실패하는 반면 이 설계는 **9큐비트**까지 올라갑니다.

가속기(Quantum MaxCut Accelerator, QMA)는 AXI IP 코어로 만들어져 RISC-V Rocket 코어 SoC에 붙고, 변분(variational) 파라미터 최적화라는 고전 루프는 하드웨어가 아니라 Rocket 코어 위의 C 코드가 담당합니다. 9큐비트 기준 소프트웨어 대비 실행시간 2,182배, 에너지 852배 개선.

여러분 프로젝트 입장에서 이 논문의 가치는 QAOA가 아니라 **에뮬레이터 하드웨어 골격**입니다. 실제로 이 논문의 참고문헌 [10]이 같은 저자(S. Choi, W. Lee)의 Grover FPGA 에뮬레이터 논문이고, 그건 여러분 폴더의 ../papers/2. Grover 논문 (LPSoC)/Developing a Grover's quantum algorithm emulator on standalone FPGAs.pdf 바로 그 논문입니다. 즉 이 두 편은 같은 연구실의 형제 논문이고, SoC 골격을 공유합니다.

목차

절	내용	원문 위치
0	읽기 전에 — 이 논문의 지도	—
1	Introduction — 왜 임베디드용 QC 에뮬레이터인가	p.1-2, §I
2	QAOA와 Weighted-MaxCut 기초	p.2-3, §II
3	에뮬레이터를 위한 최적화 전략 ★	p.3-5, §III
4	설계와 구현 ★★	p.5-7, §IV

절	내용	원문 위치
5	평가 — 시간·에너지·자원	p.7-8, §V
6	결론	p.8, §VI
7	이 프로젝트에 어떻게 쓰이나	—

★ 표시는 Grover 에뮬레이터를 만드는 입장에서 가장 값어치가 큰 절입니다.

0. 읽기 전에 — 이 논문의 지도

논문 전체가 딱 하나의 논리 사슬로 되어 있습니다. 이걸 먼저 머리에 넣고 읽으면 8쪽이 훨씬 쉬워집니다.

n 큐비트 상태벡터는 길이 $N = 2^n$ 이고, 게이트는 $N \times N$ 행렬입니다. 이걸 곧이곧대로 하드웨어에 올리면 곱셈기 개수가 $O(N^2) = O(4^n)$ 으로 폭발해서 3큐비트에서 이미 FPGA가 터집니다. → 그러니 **행렬을 그대로 두지 말고 구조를 뜯어야** 합니다. → QAOA의 두 연산자를 뜯어 보니 둘 다 아다마르 행렬 $\times$ 대각행렬 로 쓸 수 있습니다. 대각행렬 곱은 원소별 곱이라 $O(N)$ 이고, 아다마르 행렬은 원소가 전부 ± 1 이라 **곱셈이 아예 필요 없고 덧셈/뺄셈만** 하면 됩니다. → 남은 곱셈마저도 클럭당 1개씩만 하도록 파이프라인으로 흘리면, 곱셈기(DSP) 개수가 큐비트 수와 무관해집니다. → 이제 자원은 상태벡터 저장용 레지스터(FF)와 덧셈기(LUT)만 $O(N)$ 으로 늘어나므로 9큐비트까지 갑니다.

이 사슬에서 “QAOA라서” 필요한 건 사실 대각행렬 D 의 **값을 계산하는 부분**뿐입니다. 골격 자체는 알고리즘과 무관합니다. 이 점이 7절에서 Grover로 넘어가는 근거가 됩니다.

1. Introduction — 왜 임베디드용 QC 에뮬레이터인가

원문 p.1-2, §I

저자들이 세운 문제의식은 두 단계입니다.

첫째, 왜 에뮬레이터인가. 실제 양자 하드웨어는 아직 확장성과 오류정정 문제로 실용 규모에 못 미칩니다. 그 간극을 메우기 위해 고전 컴퓨터에서 양자 알고리즘을 흉내 내는 QC 에뮬레이터가 필요하고, 최근에는 비용·성능·재구성 가능성 때문에 FPGA 기반이 늘고 있습니다.

둘째, 왜 하필 standalone인가. 여기가 이 논문의 진짜 차별점입니다. 기존 FPGA 기반 QC 에뮬레이터는 거의 전부 **고성능 컴퓨팅 시스템에 붙는 가속기**로 설계되어 있습니다. 호스트 서버가 있어야 돌아간다는 뜻이죠. 그러면 QC 응용을 모바일/엣지 디바이스에 올리려는 시나리오를 지원할 수 없습니다. 그래서 저자들은 서버 없이 **FPGA 위에서 프로세서까지 통째로** 돌아가는 완결형 시스템을 목표로 잡습니다.

타깃 알고리즘으로 QAOA를 고른 이유는 조합최적화 근사해를 구하는 알고리즘이고, 특히 그래프 분할(MaxCut)이 임베디드/엣지 환경에서 실제로 쓸모가 많기 때문입니다.

핵심 난제는 “제한된 자원 안에서 최대 몇 큐비트를 지원하느냐”입니다. 큐비트 수 n 이 하나 늘 때마다 벡터·행렬 크기가 지수적으로 커지니까요. 저자들의 대응은 하드웨어-소프트웨어 코디자인이고, 구체적으로 (i) QAOA

연산식을 하드웨어 친화적으로 다시 쓰기 $\rightarrow$ 시간복잡도 $O(N^2) \rightarrow O(N)$, (ii) 그 식을 파이프라인 구조로 배치해 병렬 곱셈 개수 줄이기, (iii) 결과물을 IP 코어로 만들어 RISC-V SoC에 통합하기, 세 가지입니다.

미리 밝혀 둔 성과는 다음과 같습니다. 에너지 효율이 2큐비트 1.5배에서 9큐비트 854배까지 지수적으로 벌어지고, 중급 보드(Kintex-7)에서 기존 3큐비트 $\rightarrow$ 제안 9큐비트, 엔트리급 보드(Artix-7)에서 기존 2큐비트 $\rightarrow$ 제안 8큐비트로 확장됩니다.

(초록에서는 에너지 절감을 1.53배-852배, 본문 서론에서는 1.5배-854배로 적고 있어 숫자가 미세하게 다릅니다. 5절 Table I에서 실측치로 검증되는 값은 **1.53배 / 852배** 쪽입니다.)

2. QAOA와 Weighted-MaxCut 기초

원문 p.2-3, §II

이 절은 QAOA 이론 자체보다는 **3절의 하드웨어 최적화가 왜 가능한지**를 위한 준비 운동입니다. 뒤에서 쓸 결론만 확실히 챙기면 됩니다: (a) 비용 해밀토니안(Hamiltonian) $\hat{H}_C$ 는 **대각행렬**이다, (b) 믹서 유니터리는 R_X 회전의 텐서곱이다.

2.1 QAOA의 뼈대 (원문 II-A)

QAOA는 두 종류의 앤사츠(ansatz) 연산자 — 비용 앤사츠 $U_C(\gamma_k)$ 와 믹서 앤사츠 $U_M(\beta_k)$ — 를 번갈아가는 알고리즘입니다. 깊이 p 회로의 k 번째 레이어($k \leq p$)에서 각각 파라미터 γ_k, β_k 로 조절됩니다.

$$U_C(\gamma_k) = e^{-i\gamma_k \hat{H}_C}, \quad U_M(\beta_k) = e^{-i\beta_k \hat{H}_M} \quad (\text{원문 Eq. 1})$$

$\hat{H}_C$ 는 비용함수를 담은 **비용 해밀토니안**, $\hat{H}_M$ 은 중첩(superposition)을 유도해 탐색 공간을 넓히는 **믹서 해밀토니안**입니다.

직관적으로는 U_C 가 각 후보해에 “너는 얼마나 좋은 답이냐”를 **위상(phase)으로 새겨 넣고**, U_M 이 그 위상 차이를 **진폭(amplitude) 차이로 바꿔 섞습니다**. Grover에서 오라클이 정답에 부호를 새기고 확산 연산자가 그 부호를 진폭 크기로 바꾸는 것과 정확히 같은 패턴입니다 — 해설서 9장의 “막대 두 번 흔들기”를 떠올리면 됩니다. 차이는 Grover가 새기는 위상이 -1 로 고정인 반면 QAOA는 $e^{i\theta}$ 로 연속적이고 그 θ 를 학습한다는 점입니다.

초기 상태는 균등중첩 $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n}$ 이고, p 개 레이어를 다 걸면

$$|\psi_p(\gamma, \beta)\rangle = e^{-i\beta_p \hat{H}_M} e^{-i\gamma_p \hat{H}_C} \dots e^{-i\beta_1 \hat{H}_M} e^{-i\gamma_1 \hat{H}_C} |s\rangle \quad (\text{원문 Eq. 2})$$

변분 파라미터는 $\gamma = (\gamma_1, \dots, \gamma_p)$, $\beta = (\beta_1, \dots, \beta_p)$ 로 총 $2p$ 개입니다. 이 숫자를 기억해 두세요 — 4.3절에서 파이프라인을 “ p 레이어에 걸쳐 $2p$ 번 실행”하는 근거입니다.

최적의 (γ, β) 를 찾는 일은 **고전 컴퓨터**가 합니다. 앤사츠 상태를 반복 측정(measurement)해 $\hat{H}_C$ 의 기댓값을 구하고,

$$F_p(\gamma, \beta) = \langle \psi_p(\gamma, \beta) | \hat{H}_C | \psi_p(\gamma, \beta) \rangle = \sum_x p(x) C(x) \quad (\text{원문 Eq. 3})$$

고전 옵티마이저가 이 기댓값을 최대화하는 파라미터를 찾아 양자 쪽을 다시 실행하는 루프를 돕니다 — QAOA를 “고전-양자 하이브리드”라 부르는 이유죠. Eq. 3의 오른쪽 등호를 눈여겨보세요. **양자역학적 기댓값이 곧 고전 확률 가중평균**($p(x) = \text{해 } x \text{의 관측 확률}$)이라는 뜻이라, 하드웨어에서는 그냥 $\sum |\text{amp}|^2 C(x)$ 로 계산하면 됩니다.

원문 Fig. 1 (p.2)은 이 루프의 전체 그림입니다. 점선 위쪽이 Quantum Computer 영역으로 $|0\rangle^{\otimes n} \rightarrow H^{\otimes n}$ (Init. state) $\rightarrow U_C(\gamma_1)U_M(\beta_1)U_C(\gamma_2)U_M(\beta_2)\cdots U_C(\gamma_p)U_M(\beta_p)$ (Ansatz) $\rightarrow$ Measure로 이어지는 회로가 일렬로 그려져 있고, 점선 아래쪽 Classic Computer 영역에 “Variational Parameter Set”과 “Classical Optimization for $F_p(\gamma, \beta)$ ” 두 블록이 있어서 측정 결과를 받아 파라미터를 갱신해 다시 앤사츠 블록으로 화살표를 되돌립니다. 이 점선 하나가 4절에서 **QMA 하드웨어 / Rocket 코어 소프트웨어**의 경계선으로 그대로 구현됩니다. (그림 안 라벨이 믹서까지 $U_M(\gamma_k)$ 로 인쇄되어 있는데 본문 Eq. 1 기준으로 $U_M(\beta_k)$ 가 맞습니다 — 원문 그림의 오타입니다.)

2.2 Weighted-MaxCut 문제 (원문 II-B)

가중치가 붙은 그래프를 두 덩어리로 자르되, **잘린 간선들의 가중치 합이 최대**가 되게 나누는 문제입니다. **원문 Fig. 2 (p.2)**는 정점 6개·간선 11개짜리 예시로, 왼쪽에 원래 그래프가 있고 오른쪽에 두 색으로 음영 처리된 분할이 그려져 있으며 분할선을 가로지르는 간선들이 강조되어 있습니다 (가중치는 그림에서 생략).

각 정점 x_i 는 두 부분집합 중 하나에 속하므로 0/1 라벨을 받고, 임의의 분할은 비트열 $\mathbf{x} = x_1x_2\cdots x_{|V|}$ 하나로 표현됩니다. 비용함수는

$$C(\mathbf{x}) = \sum_{i,j=1}^{|V|} \omega_{ij} x_i(1 - x_j) \quad (\text{원문 Eq. 4})$$

두 정점이 같은 집합이면 $x_i(1 - x_j) = 0$ 이라 그 간선은 기여하지 않고 다른 집합이면 기여합니다 — **“잘린 간선만 세는”** 식입니다. 문제는 $C(\mathbf{x})$ 를 최대화하는 $\mathbf{x}$ 를 찾는 것이 됩니다.

이제 QAOA에 태우기 위해 두 해밀토니안을 정의합니다.

$$\hat{H}_C = \frac{1}{2} \sum_{(i,j) \in E} \omega_{ij} (I - Z_i Z_j), \quad \hat{H}_M = \sum_{j \in V} X_j \quad (\text{원문 Eq. 5})$$

Z_i 는 i 번째 큐비트에 대한 파울리-Z 게이트(gate)입니다. $Z_i Z_j$ 항의 l 번째 대각원소는

$$Z_i Z_j(l, l) = \begin{cases} 1 & \text{if the } l\text{-th bits of } i \text{ and } j \text{ are different} \\ -1 & \text{if the } l\text{-th bits of } i \text{ and } j \text{ are the same} \end{cases} \quad (\text{원문 Eq. 6})$$

여기서 이 논문에서 가장 중요한 독해 주의사항이 나옵니다. Eq. 6의 문장은 l 과 (i, j) 의 역할이 뒤바뀌어 인쇄되어 있습니다. 실제로 의미가 통하는 해석은 “인덱스 $l - 1$ 의 i 번째 비트와 j 번째 비트가 다른가 같은가”입니다. 부호와 계수도 Eq. 5의 $\frac{1}{2}$, Eq. 7의 $-2\omega_{ij}\gamma_k$, 그리고 4.2절 실제 코드의 $2.0 * \text{weight}$ 가 서로 2배씩 어긋나 보입니다. 구현할 때는 **4.2절의 C 코드를 기준으로** 삼으세요. 실용적으로는 이 상수배가 문제되지 않는데, γ 가 어차피 최적화로 학습되는 변분 파라미터라 상수배는 γ 의 스케일에 흡수되고, I 항이 만드는 전역 위상은 측정 결과를 바꾸지 않기 때문입니다.

$\hat{H}_C$ 가 **대각행렬**이라는 점을 붙잡아 두세요. I 도 대각, $Z_i Z_j$ 도 대각이니 그 부분합인 $\hat{H}_C$ 도 대각입니다. 3절 전체가 이 사실 위에 세워집니다.

해밀토니안을 유니터리 게이트로 바꾸면 k 번째 레이어의 두 연산은 이렇게 됩니다.

$$U_C(\gamma_k) = e^{-i\gamma_k \hat{H}_C} = \prod_{i=1, j < i}^n R_{Z_i Z_j}(-2\omega_{ij}\gamma_k) \quad (\text{원문 Eq. 7})$$

$$U_M(\beta_k) = e^{-i\beta_k \hat{H}_M} = \prod_{i=1}^n R_{X_i}(2\beta_k) \quad (\text{원문 Eq. 8})$$

Eq. 8이 말하는 건 간단합니다. 믹서는 **모든 큐비트에 똑같은 $R_X(2\beta_k)$ 회전을 동시에 거는 것**뿐입니다. 이 단순함이 3.1절에서 대박을 칩니다.

3. 에뮬레이터를 위한 최적화 전략 ★

원문 p.3-5, §III

문제 제기는 명확합니다. QAOA를 에뮬레이션하려면 U_C, U_M 유니터리 행렬을 상태벡터에 반복해 곱해야 하는데, n 큐비트면 $N \times N$ 행렬 $\times N \times 1$ 벡터라 $O(N^2) = O(2^{2n})$ 짜리 레지스터와 연산력이 듭니다. SoC의 제한된 자원으로는 답이 없죠. 그래서 (1) 믹서 엔사츠의 행렬 표현을 **분해**해 전체를 하드웨어 최적화된 기본 연산(elemental operation)으로 재구성하고, (2) 그 기본 연산을 **파이프라인**에 태워 병렬 곱셈 수요를 줄입니다.

3.1 믹서 유니터리 분해 (원문 III-A)

$R_X(2\beta_k)$ 의 행렬 표현부터 시작합니다.

$$R_X(2\beta_k) = e^{-i\beta_k X} = \cos(\beta_k)I - i\sin(\beta_k)X = \begin{bmatrix} \cos(\beta_k) & -i\sin(\beta_k) \\ -i\sin(\beta_k) & \cos(\beta_k) \end{bmatrix} \quad (\text{원문 Eq. 9})$$

여기서 저자들의 관찰이 나옵니다. I 와 X 는 **둘 다** $\mathbf{v}_1 = [1 \ 1]^T, \mathbf{v}_2 = [1 \ -1]^T$ 를 고유벡터로 가집니다. R_X 가 그 둘의 선형결합이니 R_X 도 같은 고유벡터를 가지고, 고깃값만 계산하면 됩니다.

$$R_X(2\beta_k)\mathbf{v}_1 = (\cos \beta_k - i \sin \beta_k) \begin{bmatrix} 1 \\ 1 \end{bmatrix} = e^{-i\beta_k}\mathbf{v}_1 = \lambda_1\mathbf{v}_1 \quad (\text{원문 Eq. 10})$$

$$R_X(2\beta_k)\mathbf{v}_2 = (\cos \beta_k + i \sin \beta_k) \begin{bmatrix} 1 \\ -1 \end{bmatrix} = e^{i\beta_k}\mathbf{v}_2 = \lambda_2\mathbf{v}_2 \quad (\text{원문 Eq. 11})$$

$\lambda_1 = e^{-i\beta_k}$, $\lambda_2 = e^{i\beta_k}$. 그런데 $\mathbf{v}_1, \mathbf{v}_2$ 를 열로 세운 행렬이 정확히 **아다마르 행렬**입니다. 그래서 고윳값 분해가 이렇게 떨어집니다.

$$R_X(2\beta_k) = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} e^{-i\beta_k} & 0 \\ 0 & e^{i\beta_k} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}^{-1} = H\Lambda_M H \quad (\text{원문 Eq. 12})$$

(H 가 자기 자신의 역행렬이라 $H^{-1} = H$ 로 정리됩니다.)

이제 Eq. 8에서 U_M 은 n 개 큐비트에 R_X 를 동시에 거는 것이었으므로 텐서곱으로 쓰면,

$$U_M(\beta_k) = R_X(2\beta_k)^{\otimes n} = (H\Lambda_M H)^{\otimes n} = H^{\otimes n} D_M H^{\otimes n} \quad (\text{원문 Eq. 13})$$

여기서 $D_M = \Lambda_M^{\otimes n}$ 은 믹서 엔사츠를 나타내는 대각행렬입니다. 텐서곱이 곱셈에 대해 분배되기 때문에 $(H\Lambda H)^{\otimes n}$ 이 $H^{\otimes n} D_M H^{\otimes n}$ 으로 깔끔하게 갈라지는 게 핵심입니다.

한편 $\hat{H}_C$ 는 대각행렬이었으므로 그 지수함수인 $U_C(\gamma_k)$ 도 대각행렬입니다. 이걸 $D_C = U_C(\gamma_k)$ 라 두면, k 번째 레이어의 엔사츠 연산은

$$U_M(\beta_k)U_C(\gamma_k) = (H^{\otimes n} D_M)(H^{\otimes n} D_C) \quad (\text{원문 Eq. 14})$$

이것이 논문의 첫 번째 결정타입니다. 한 레이어의 연산이 $U_A = H^{\otimes n} D$ 라는 **기본 엔사츠 연산(elemental ansatz operation)** 두 번으로 쪼개졌습니다 — 비용용으로 한 번, 믹서용으로 한 번. 대각행렬 연산의 계산량은 $O(N)$ 이라 원래 행렬-벡터 곱의 $O(N^2)$ 보다 압도적으로 쌉니다.

여기서 한 번 더 짜냅니다. $H^{\otimes n}$ 의 모든 원소는 절댓값이 $1/\sqrt{2^n}$ 입니다. 그럼 스케일을 걷어내서 모든 원소의 절댓값이 **정확히 1** 인 수정 아다마르 행렬 H_1 을 정의하면,

$$H_1 = \frac{1}{\sqrt{2^n}} H^{\otimes n}, \quad \text{또는} \quad H^{\otimes n} = \sqrt{2^n} H_1 \quad (\text{원문 Eq. 15})$$

$$U_M(\beta_k)U_C(\gamma_k) = \frac{1}{2^n} (H_1 D_M)(H_1 D_C) \quad (\text{원문 Eq. 16})$$

($\sqrt{2^n}$ 이 두 번 나오니 2^n 이 되어 앞으로 빠집니다.)

이것이 두 번째 결정타입니다. H_1 의 원소는 +1 아니면 -1 뿐이므로, 아다마르 변환은 곱셈-누산(MAC)이 아니라 병렬 덧셈/뺄셈만으로 구현됩니다. 하드웨어에서 “+1 을 곱한다”는 건 그냥 통과, “-1 을 곱한다”는 건 부호 반전 회로 하나입니다. 그리고 앞에 남은 $1/2^n$ 스케일 팩터는 2의 거듭제곱이므로 비트 시프트 한 번으로 끝납니다. 곱셈기가 통째로 사라졌습니다.

3.2 비용·믹서 대각행렬의 원소 일반식 (원문 III-B)

이제 D_C 와 D_M 의 대각원소를 닫힌 형태로 구해야 합니다. 행렬 거듭제곱을 실제로 돌리는 대신 공식 하나로 뽑아내는 게 목적입니다.

D_C 부터. 행렬 지수함수의 정의로 전개하면

$$D_C = U_C(\gamma_k) = e^{-i\gamma_k \hat{H}_C} = \sum_{m=0}^{\infty} \frac{(-i\gamma_k)^m}{m!} \hat{H}_C^m \quad (\text{원문 Eq. 17})$$

$\hat{H}_C$ 가 대각이므로 거듭제곱의 대각원소는 그냥 원소의 거듭제곱입니다.

$$\hat{H}_C^m(l, l) = (\hat{H}_C(l, l))^m \quad (\text{원문 Eq. 18})$$

이 단순한 성질 덕분에 무한급수가 스칼라 지수함수로 접힙니다.

$$D_C(l, l) = \sum_{m=0}^{\infty} \frac{(-i\gamma_k \hat{H}_C(l, l))^m}{m!} = e^{-i\gamma_k \hat{H}_C(l, l)} \quad (\text{원문 Eq. 19})$$

즉 행렬 지수함수가 스칼라 지수함수 N 개로 분해됩니다. 대각원소 하나당 $e^{-i\theta}$ 한 번, 그게 전부입니다.

D_M 은 조금 더 재밌습니다. Eq. 12-13에서 $D_M = \Lambda_M^{\otimes n}$ 이고 $\Lambda_M = \text{diag}(\lambda_1, \lambda_2)$ 였습니다. 텐서곱의 대각원소는 각 인자의 대각원소를 골라 곱한 것이므로, D_M 의 l 번째 대각원소는 λ_1 과 λ_2 를 각각 몇 번씩 곱한 결과입니다. 몇 번씩이나 하면 $-l-1$ 의 이진 표현에서 1의 개수, 즉 해밍 무게(Hamming weight) $HW(l-1)$ 만큼 λ_2 를, 나머지 $n - HW(l-1)$ 만큼 λ_1 을 곱합니다.

$$D_M(l, l) = \lambda_1^{n-HW(l-1)} \times \lambda_2^{HW(l-1)} = e^{i u(n, l) \beta_k}, \quad u(n, l) = 2 \cdot HW(l-1) - n \quad (\text{원문 Eq. 20})$$

($\lambda_1 = e^{-i\beta_k}$, $\lambda_2 = e^{+i\beta_k}$ 를 대입하면 지수가 $-(n-HW) + HW = 2HW - n$ 으로 합쳐집니다.)

하드웨어 관점에서 이게 왜 대단하냐면, D_M 의 대각원소 전체가 1의 개수를 세는 회로(popcount) 하나 + 시프트/뺄셈으로 나온다는 뜻이기 때문입니다. 곱셈도, 메모리 테이블도 필요 없습니다. 실제로 4.3절 CALCULATE_RAD 스테이지에 1's counter와 $\times 2 - n$ 블록이 딱 그것으로 들어갑니다.

두 식을 나란히 놓고 보면 구조가 같습니다. 둘 다 각도(radian)를 하나 계산한 뒤 $e^{i\theta}$ 를 취하는 것입니다. $-D_C$ 는 각도가 $-\gamma_k \hat{H}_C(l, l)$, D_M 은 $u(n, l)\beta_k$. 그래서 하드웨어는 두 경우를 같은 파이프라인으로

처리하고 입력만 먹스(mux)로 골라 주면 됩니다. 4.3절의 order 레지스터가 바로 그 먹스 선택 신호입니다. (사소한 표기 주의: 원문은 Eq. 17의 합 지수를 k 로 인쇄했지만 γ_k 의 k 는 레이어 번호라 m 이 맞습니다. 또 Eq. 18-20에서 $l = 1, 2, \dots, n$ 이라 적었는데 행렬이 $N \times N$ 이므로 $l = 1, \dots, N$ 이 맞습니다. 실제 코드도 NUM_STATE = N 까지 돕니다.)

3.3 기본 앤사츠 연산의 파이프라인화 (원문 III-C)

여기까지로 $O(N)$ 은 달성했지만 문제가 하나 남았습니다. 대각행렬-벡터 곱을 **병렬로** 하려면 여전히 곱셈기 N 개가 동시에 필요합니다. 곱셈은 덧셈보다 자원을 훨씬 많이 먹으니, $N = 2^n$ 개를 병렬로 깔면 결국 지수 폭발입니다.

해법: 곱셈은 직렬로, 덧셈은 병렬로. 대각행렬 연산을 아다마르 연산과 **함께 파이프라인**에 흘려서, 클럭당 곱셈 1개 + 덧셈 N 개를 하도록 만듭니다.

원문 Fig. 3 (p.4) 이 2큐비트($N = 4$) 예시로 이 흐름을 보여 줍니다. H_1 (4×4, 원소 전부 ± 1), D (대각 $\lambda_1 \dots \lambda_4$), 입력 $|x\rangle$, 출력 $|\psi\rangle$ 를 한 줄에 놓고 클럭이 흐르는 스냅샷을 세로로 쌓았습니다. 1클럭째(1st op)는 λ_1 과 x_1 만 강조된 채 1_MULT 곱셈 한 번($\lambda_1 x_1$)만 일어나고 $|\psi\rangle$ 는 아직 $[0, 0, 0, 0]$ 입니다. 2클럭째(2nd op)에는 1_MULT 와 N_ADD 가 **동시에** 표시되는데, N_ADD 가 방금 나온 $\lambda_1 x_1$ 을 H_1 의 **1번 열** (전부 +1) 부호대로 출력 4개 성분에 모두 더해 $|\psi\rangle = [0 + \lambda_1 x_1, \dots]$ 을 만드는 **같은 클럭에** 곱셈기는 벌써 $\lambda_2 x_2$ 를 계산합니다. 다음 스냅샷에서는 H_1 의 **2번 열** (1, -1, 1, -1) 이 강조되어 $\lambda_2 x_2$ 가 부호대로 누산되고 곱셈기는 $\lambda_3 x_3$ 으로 넘어갑니다.

핵심은 $\lambda_l x_l$ 하나가 H_1 의 l 번째 “열” 전체에 기여한다는 사실입니다. 그래서 곱셈 결과 하나가 나오면 N 개 누산기에 부호만 바꿔 뿌리면 됩니다 — 곱셈 1개, 덧셈 N 개. 이 패턴이 N 개 원소 전체에 반복되어 총 $N + 1$ 클럭에 앤사츠 상태 하나가 완성됩니다 (+1 은 마지막 곱셈 결과를 누산하는 클럭).

이 파이프라인이 주는 것은 두 가지입니다. 시간복잡도가 $O(N^2) \rightarrow O(N)$ 으로 내려가고, 동시에 **병렬 곱셈기 개수가 큐비트 수와 무관하게 상수**로 묶입니다. 5절 Table II에서 DSP 사용량이 $n = 2$ 부터 $n = 10$ 까지 31-32개로 불박이인 게 정확히 이 설계의 결과입니다. 실행시간을 줄이면 자원이 늘어나는 게 보통인데, 곱셈 개수를 통제함으로써 그 트레이드오프를 관리 가능한 수준으로 눌렀다는 게 저자들의 주장입니다.

4. 설계와 구현 ★★

원문 p.5-7, §IV

여러분 프로젝트에 가장 직접적으로 베껴 쓸 수 있는 절입니다.

4.1 SoC 아키텍처와 설계 자동화 (원문 IV-A)

원문 Fig. 4 (p.5) 가 전체 시스템 다이어그램입니다. 그림 아래쪽에 좌우로 긴 **System Interconnect** 버스가 깔려 있고, 그 위에 블록들이 세로로 매달려 있습니다. 왼쪽부터 **Rocket RISC-V Core**, **SRAM System Memory**, 그리고 파란색으로 강조된 **Quantum MaxCut Accelerator (QMA)** 가 각각 **AXI** 로 붙고, 오른쪽에는 Peripherals 묶음(IROM, JTAG, FLASH 한 그룹 / SPI, I2C, UART 한 그룹)이 **APB** 로 붙습니다. 서버도 호스트 PC도 없이 이 한 장이 시스템 전부라는 점 — 그게 “standalone”의 의미입니다.

구성 요소를 정리하면 다음과 같습니다.

구성 요소	내용
프로세서	RISC-V Rocket 코어 (부동소수점 연산 유닛 포함)
메인 메모리	512KB SRAM
비휘발성 저장	Flash
인터커넥트	경량 Network-on-Chip (μ NoC), 가속기·메모리는 AXI / 주변장치는 APB
제어	boot-reset 담당 컨트롤 모듈
I/O	UART, SPI, I2C
가속기	QMA — 고정소수점(fixed-point) 연산, AXI 인터페이스 IP 코어
동작 주파수	100MHz
RTL 생성 도구	RISC-V eXpress (RVX) 를 확장한 QEX

Rocket 코어를 고른 이유는 부동소수점 유닛이 있어서 임베디드 환경의 QAOA 에뮬레이션(특히 고전 최적화 루프)에 적합하기 때문입니다. QMA는 반대로 **고정소수점**을 씁니다 — 하드웨어 연산에 적합하다는 이유요. 즉 이 시스템은 **부동소수점 CPU + 고정소수점 가속기**의 조합입니다. 다만 논문은 QMA의 **비트 폭이나 Q-포맷을 끝까지 밝히지 않습니다**. 정밀도가 결과에 어떤 영향을 주는지도 다루지 않고요. 이걸 이 논문의 명백한 공백이고, 여러분 폴더의 ../papers/2. Grover 논문 (LPSoC)/Precision-aware fixed-point emulation of Grover's algorithm.pdf 가 정확히 그 공백을 메우는 논문입니다. 두 편을 짝지어 읽으세요.

QMA가 **AXI IP 코어**로 만들어졌다는 점이 실용적으로 중요합니다. 어떤 프로세서 플랫폼에도 그대로 붙일 수 있고, 실제로 5절에서 Rocket(Kintex-7)과 ORCA(Artix-7) 두 개의 서로 다른 코어에 같은 QMA를 붙여 실험합니다.

설계 자동화 (QEX). 큐비트 수마다 RTL을 손으로 고치는 건 말이 안 되니, RVX를 확장해 **QC Emulator eXpress (QEX)** 를 만들었습니다. 동작은 이렇습니다.

1. 사용자가 파라미터 NUM_QUBIT 을 준다.
2. QEX가 기저 상태 계수와 대각행렬 원소를 담은 **레지스터 블록** 개수를 계산해 로컬 파라미터 NUM_STATE ($= 2^{\text{NUM_QUBIT}}$) 로 정의한다.
3. 그에 맞춰 MaxCut 가속기의 레지스터와 연산 유닛을 구성하고, Fig. 4의 SoC 플랫폼 RTL 전체를 생성한다.

여기서 반드시 짚어야 할 게 있습니다. 상태벡터를 “**레지스터 블록**”에 저장한다고 명시되어 있고, 5절의 자원 표에도 **BRAM 열이 아예 없습니다** (LUT/FF/DSP만 보고). 즉 이 설계는 상태벡터를 BRAM이 아니라 **플립플롭(FF)** 에 통째로 올립니다. 왜냐면 N_ADD 스테이지가 매 클럭 N 개 누산기를 **동시에** 갱신해야 하는데, BRAM은 포트가 2개뿐이라 그게 불가능하기 때문입니다. 이 선택이 성능의 원천이자 동시에 확장의 벽입니다 — 9큐비트에서 FF 94,978개, 10큐비트에서 163,157개(40%)를 먹고 LUT가 113%로 터집니다.

C API. QMA를 다루는 C 언어 API 네 개를 제공합니다.

명령	역할
set_parameter	QAOA 레이어 번호(layer)와 γ, β 를 고정소수점으로 설정

명령	역할
set_cost_hamiltonian	간선 (i, j) 와 가중치를 입력해 비용 해밀토니안 구성
activate_maxcut	에뮬레이션 실행
get_expectation	앤사츠 상태의 기댓값 회수

동작 흐름은 이 API로 짠 응용 프로그램을 SRAM에 올려 Rocket 코어가 실행하고, 에뮬레이션이 끝나면 MaxCut 가속기가 기댓값을 Rocket 코어에 돌려주고, **사용자(=Rocket 코어의 소프트웨어)가 그 결과로 변분 파라미터를 최적화**하는 것입니다.

고전-양자 하이브리드 루프가 실제로 어떻게 처리되는가 — 이게 여러분이 궁금해할 부분일 텐데, 답은 명확합니다. **하드웨어는 상태 전개만 하고, 최적화 루프는 하지 않습니다.** QMA는 “파라미터 받아서 → 상태 굴리고 → 기댓값 뱉기”까지만 하는 순수 함수이고, 그 기댓값을 보고 γ, β 를 어떻게 바꿀지 결정하는 옵티마이저는 Rocket 코어 위의 C 코드입니다. Fig. 1의 점선(Quantum / Classic 경계)이 그대로 Fig. 4의 QMA / Rocket 경계가 된 셈이죠. 참고로 논문은 **어떤 고전 최적화 알고리즘을 썼는지 명시하지 않고**, 5절 평가도 “8-레이어 QAOA 1회 실행” 시간만 재지 옵티마이저가 몇 번 돌아 수렴했는지는 측정하지 않습니다. 이것도 이 논문의 공백입니다.

4.2 QAOA 실행 준비 (원문 IV-B)

QAOA를 돌리기 전에 비용 해밀토니안을 정의해야 합니다. set_cost_hamiltonian 으로 각 간선이 있는 두 정점 i, j 와 가중치 weight (고정소수점)를 QMA에 넣으면, Eq. 5·6에 따라 $Z_i Z_j$ 부분함으로 해밀토니안이 구성되고 대각원소 cost_hamil_diag 가 다음 알고리즘으로 계산됩니다 (원문 p.5).

```
for (k = 0; k < NUM_STATE; k++) {
    bit_i = (k >> (i-1)) & 1;
    bit_j = (k >> (j-1)) & 1;
    if (bit_i != bit_j) cost_hamil_diag[k] += 2.0 * weight;
}
```

이 8줄이 2.2절에서 얘기한 Eq. 6 혼란의 **정답지**입니다. 읽는 법: 기저 상태 인덱스 k 를 하나 잡고, 그 k 의 i 번째 비트와 j 번째 비트를 꺼내서, 두 비트가 **다르면**(= 두 정점이 서로 다른 집합에 있으면 = 이 간선이 잘렸으면) 가중치를 더합니다. 간선마다 이 루프를 한 번씩 돌면서 += 로 누적하면 됩니다. 딱 Eq. 4의 $\omega_{ij} x_i (1 - x_j)$ 를 모든 기저 상태에 대해 미리 계산해 테이블로 만들어 두는 것이고, **간선 개수 $\times N$ 번**이면 끝나므로 비싸지 않습니다.

set_parameter 는 레이어 번호와 γ, β 를 고정소수점으로 gamma[layer], beta[layer] 레지스터에 씁니다. NUM_QUBIT 은 합성 시점에 정해지는 파라미터로, 이 에뮬레이터가 다룰 수 있는 Weighted-MaxCut 문제의 크기를 결정합니다.

4.3 레지스터 수준 파이프라인 (원문 IV-C) ★

3.3절 파이프라인은 아직 반쪽입니다. D_C, D_M 의 대각원소를 Eq. 17·20으로 구하면 나오는 건 **극형식의 지수부, 즉 각도값**입니다. 하드웨어에서 실제로 곱셈을 하려면 이것 **직교형식(cos, sin)** 으로 바꿔야 하고,

그러려면 삼각함수 계산이 필요합니다. 저자들은 3.3절 파이프라인 앞에 **CORDIC 모듈**을 스테이지로 이어 붙여 완전한 파이프라인을 완성했습니다.

원문 Fig. 5 (p.6) 는 이 파이프라인의 레지스터 수준 회로도, 페이지 폭을 다 쓰는 가로로 긴 그림입니다. 왼쪽부터 오른쪽으로 **CALCULATE_RAD** → **NORMALIZE_RAD** → **CORDIC** → **1_MULT** → **N_ADD** 다섯 칸이 세로 구분선으로 나뉘어 있고, 각 칸 사이에 파이프라인 레지스터(회색 사다리꼴)가 그려져 있습니다. 왼쪽 끝에는 beta/gamma 레지스터 파일과 1's counter, cost_hamil_diag 레지스터 배열([0]부터 $[N - 1]$ 까지)이 있고, layer_order 신호가 맥스를 제어해 곱셈기($\otimes$)로 들어가 rad 를 만듭니다. 오른쪽 끝에는 result[0..N-1] 누산기 배열이 각각 덧셈기($\oplus$)를 달고 있으며, 굵은 회색 화살표가 result 에서 state[0..N-1] 로 되돌아가 다음 기본 연산의 입력이 됨을 나타냅니다.

이 파이프라인은 p 레이어에 걸쳐 $2p$ 번 실행됩니다 (레이어당 비용 1번 + 믹서 1번). 그래서 지금이 비용인지 믹서인지 알려 주는 레지스터 order 와, 현재 레이어를 가리키는 layer 두 개로 제어합니다.

스테이지 1: CALCULATE_RAD D_C, D_M 각 대각원소의 라디안 각도를 Eq. 17·20으로 계산합니다. activate_maxcut 이 실행되면 파이프라인은 NUM_STATE 클럭 동안 돕니다. 카운터 count_1st 가 지금 몇 번째 대각원소를 처리 중인지 가리킵니다.

```
if (!order) { rad = cost_hamil_diag[count_1st] * gamma[layer]; }
else      { rad = u(count_1st, NUM_QUBIT) * beta[layer]; }
```

order 가 0(비용)이면 미리 만들어 둔 cost_hamil_diag[count_1st] 에 gamma[layer] 를 곱하고, 1(믹서)이면 $u(n, l) = 2 \cdot HW(l - 1) - n$ 에 beta[layer] 를 곱합니다. Fig. 5에서 1's counter → $\times 2 - n$ 블록이 바로 이 u 계산이고, **비용 경로는 메모리 룩업, 믹서 경로는 popcount 산술**이라는 비대칭이 그림에 그대로 보입니다. 결과는 rad 레지스터에 저장되어 다음 스테이지로 흘러갑니다. 곱셈기가 하나 필요한 지점입니다.

스테이지 2: NORMALIZE_RAD CORDIC이 정확히 동작하려면 입력 각도가 **1사분면에** 있어야 합니다. 그래서 각도를 2π 로 모듈로 연산한 뒤, 원래 각도가 있던 사분면에 따라 1사분면 각도로 접고 부호를 따로 채깁니다.

```
neg_cos = 0; neg_sin = 0; norm_rad = rad % (2 * PI);
if (en_pipeline) {
    if (norm_rad < 0.5 * PI) norm_rad_first_quad = norm_rad;
    else if (norm_rad < PI) { norm_rad_first_quad = PI - norm_rad; neg_cos = 1; }
    else if (norm_rad < 1.5 * PI) {
        norm_rad_first_quad = norm_rad - PI;
        neg_sin = 1; neg_cos = 1; }
    else {
        norm_rad_first_quad = 2 * PI - norm_rad; neg_sin = 1; }
}
```

정규화된 각도는 radQ1 레지스터에, 사분면 때문에 뒤집힌 삼각함수 부호는 **sign adjustment 레지스터**(neg_cos, neg_sin)에 저장됩니다. 익숙한 $\cos(\pi - \theta) = -\cos \theta$ 류의 대칭성을 하드웨어로 옮긴 것뿐입니다. 이렇게 하면 CORDIC이 $[0, \pi/2]$ 만 커버하면 되므로 회로가 작아집니다.

스테이지 3: CORDIC radQ1 의 $\cos$ 와 $\sin$ 을 동시에 계산합니다. 16단 파이프라인으로 구현되어 입력 각도가 16클럭에 걸쳐 통과하고, 끝나면 $\cos Q1$, $\sin Q1$ 과 완료를 알리는 valid 신호가 나옵니다. CORDIC은 곱셈기 없이 시프트-덧셈만으로 삼각함수를 구하는 고전적 기법이라([31] Volder, 1959) 이 설계 철학과 잘 맞습니다.

중요한 디테일: 스테이지 2에서 만든 **부호 조정값들도 CORDIC과 나란히 16단을 함께 흘러갑니다**. 같은 클럭에 나오는 삼각함수 값과 부호가 짝이 맞아야 하니까요. Fig. 5에 sign adj. 와 count_4th 가 CORDIC 블록 옆으로 병렬 통과하는 것으로 그려져 있습니다. 파이프라인 설계에서 흔히 놓치는 지점이라 눈여겨볼 만합니다.

스테이지 4: 1_MULT CORDIC이 뱀은 대각원소를 상태벡터의 해당 성분과 곱합니다 (Fig. 3의 1_MULT). 카운터 count_4th 가 상태벡터에서 $\text{state}[\text{count_4th}]$ 를 골라 옵니다. **대각원소도 상태벡터 성분도 복소수이므로 실수부·허수부를 나눠 계산합니다**.

$$\begin{aligned}\text{Re}(\text{mult}) &= \text{Re}(\text{state}[\text{count_4th}]) * \cos_theta \\ &\quad - \text{Im}(\text{state}[\text{count_4th}]) * \sin_theta; \\ \text{Im}(\text{mult}) &= \text{Re}(\text{state}[\text{count_4th}]) * \sin_theta \\ &\quad + \text{Im}(\text{state}[\text{count_4th}]) * \cos_theta;\end{aligned}$$

복소수 곱셈 하나 = **실수 곱셈 4개 + 덧셈 2개**입니다. 이게 클럭당 한 번, 큐비트 수와 무관하게 딱 한 세트. Table II에서 DSP가 31-32개로 고정인 이유의 절반이 여기 있습니다(나머지 절반은 CORDIC과 스테이지 1의 곱셈기).

스테이지 5: N_ADD 1_MULT 결과를 H_1 의 해당 열 원소들과 곱해서 result 레지스터에 누산합니다 (Fig. 3의 N_add). H_1 원소는 ± 1 뿐이므로, 곱셈은 **단순 부호 반전 로직 + 병렬 덧셈**으로 구현됩니다.

```
for (i = 0; i < NUM_STATE; i++) {
    if (H_1[i][count_5th] == 1) result[i] += mult;
    else if (H_1[i][count_5th] == -1) result[i] -= mult;
    else printf("Error: Unexpected value in H_1");
}
```

이 for 루프는 소프트웨어의 순차 반복이 아니라 **하드웨어의 완전 병렬 전개**입니다. NUM_STATE 개의 덧셈기가 동시에 동작하고, 한 클럭에 다 끝납니다. 전체 파이프라인에서 기본 앤사츠 연산 하나가 끝나면 state 를 result 의 누산값으로 재초기화하고 다음 연산으로 넘어갑니다.

정리: 타이밍 계산 파이프라인 전체를 시간으로 환산해 보면 (아래는 논문에 없는 필자의 추정 계산이니 참고용입니다):

- 기본 앤사츠 연산 1회 = 약 $N + 1$ 클럭 + 파이프라인 채우기 지연(CORDIC 16 + 나머지 약 4단)
- 레이어당 2회 (비용 + 믹서), p 레이어면 총 $2p$ 회
- $n = 9(N = 512)$, $p = 8$ 이면 $\approx 16 \times (512 + 20) \approx 8,500$ 클럭 = 100MHz에서 **약 $85\mu\text{s}$**

Table I의 9큐비트 실측치가 0.34ms이니, 나머지 약 0.25ms는 $\text{set_cost_hamiltonian}$ 구성, API 오버헤드, 기댓값 계산 등 Rocket 코어 쪽 일이 차지한다고 볼 수 있습니다. 즉 **9큐비트에서도 가속기 자체보다 소프트웨어 오버헤드가 더 큼**니다. 여러분이 Grover IP의 성능을 추정할 때도 같은 함정을 염두에 두세요 — 코어 파이프라인만 빨라도 전체는 그만큼 안 빨라집니다.

5. 평가 — 시간·에너지·자원

원문 p.7-8, §V

실험 설정. QEX로 큐비트 수를 바꿔 가며 에뮬레이터를 만들고, Xilinx Vivado 로 합성해 Xilinx Kintex-7 XC7K325T 보드에 프로토타이핑했습니다. 동작 확인 후 **8-레이어 QAOA**의 실행 시간을 측정했고, 에너지는 실측 실행시간과 Vivado가 보고한 전력값으로 계산했습니다. 비교군은 QMA만 빼고 **아키텍처가 동일한 베이스라인 프로세서**로, 같은 보드에 올려 QAOA를 C 소프트웨어로 실행했습니다.

원문 Fig. 6 (p.7) 은 실물 사진 두 장입니다. 왼쪽은 Kintex-7 FPGA 보드에서 Rocket 기반 에뮬레이터(w/ QMA)가 돌아가는 모습, 오른쪽은 Artix-7 보드에서 ORCA 기반 에뮬레이터가 돌아가는 모습이고, 각각 노트북 화면에 실행 로그가 떠 있습니다.

5.1 실행 시간과 에너지 (원문 Table I, p.7)

8-레이어 QAOA 에뮬레이션 기준, QMA 유무 비교입니다.

n	w/o QMA 실행시간 (ms)	w/o QMA 에너지 (mJ/exec.)	제안 실행시간 (ms)	제안 에너지 (mJ/exec.)
2	0.41	0.23	0.26	0.15
3	0.98	0.54	0.26	0.15
4	2.1	1.2	0.26	0.15
6	16	8.6	0.27	0.17
8	196	108	0.30	0.30
9	742	409	0.34	0.48

이 표에서 읽어야 할 건 **두 열의 증가 속도 차이**입니다. 소프트웨어(w/o QMA)는 n 이 2에서 9로 가는 동안 $0.41\text{ms} \rightarrow 742\text{ms}$ 로 **약 1,800배** 폭증합니다 ($O(N^2)$ 이니 큐비트 하나당 4배씩). 반면 제안 설계는 $0.26\text{ms} \rightarrow 0.34\text{ms}$ 로 **거의 평평합니다**. $O(N)$ 이라 원래는 큐비트당 2배씩 늘어야 하지만, 4.3절에서 계산했듯 이 구간에서는 가속기 시간이 전체의 소수라 소프트웨어 오버헤드에 가려집니다.

논문이 강조하는 수치는 이렇습니다.

- **2큐비트:** 실행시간 약 **1.56배**, 에너지 효율 **1.53배** 개선. (작은 문제에선 이득이 미미)
- **9큐비트:** 속도 **2,182배**, 에너지 효율 **852배** 개선.

즉 **이득이 큐비트 수에 따라 지수적으로 벌어집니다**. 에너지가 852배인데 속도가 2,182배인 건 QMA를 넣으면 전력 자체는 올라가기 때문입니다(제안 설계의 에너지는 $0.15 \rightarrow 0.48\text{mJ}$ 로 늘고 있죠). 그래도 시간이 훨씬 더 많이 줄어드니 총 에너지는 크게 이깁니다.

5.2 자원 사용량 — Kintex-7 (원문 Table II, p.7) ★

이 표가 논문의 하이라이트입니다. **Conventional** 은 저자들의 최적화 없이 범용 QC용 행렬-벡터 곱 가속기를 그냥 만든 것으로, Fig. 4 플랫폼에서 QMA 자리에 끼워 넣고 큐비트 수마다 RTL을 조정했습니다.

기존 방식 가속기 (Conventional) — Kintex-7 XC7K325T, Rocket 코어

n	LUT (사용률)	FF (사용률)	DSP (사용률)
	(Max: 203,800)	(Max: 407,600)	(Max: 840)
2	43,086 (21.1%)	26,814 (6.6%)	103 (12.3%)
3	136,437 (66.9%)	30,412 (7.5%)	315 (37.5%)
4 (실패)	1,275,722 (626%)	41,146 (10.1%)	834 (99.3%)

제안 QMA — 동일 보드/코어

n	LUT (사용률)	FF (사용률)	DSP (사용률)
	(Max: 203,800)	(Max: 407,600)	(Max: 840)
2	28,999 (14.2%)	27,011 (6.6%)	31 (3.7%)
3	29,486 (14.5%)	27,524 (6.8%)	31 (3.7%)
4	30,510 (15.0%)	28,597 (7.0%)	31 (3.7%)
6	36,369 (17.8%)	34,813 (8.5%)	32 (3.8%)
8	79,252 (38.9%)	60,724 (14.9%)	32 (3.8%)
9	130,134 (63.9%)	94,978 (23.3%)	32 (3.8%)
10 (실패)	230,183 (113.0%)	163,157 (40.0%)	32 (3.8%)

참조값: QC 가속기가 없는 순수 에뮬레이터의 LUT/FF/DSP = **24,853 / 24,445 / 4**

이 두 표에서 뽑아야 할 결론이 네 가지입니다.

첫째, DSP가 상수입니다. 기존 방식은 103 → 315 → 834개로 큐비트 하나당 3배씩 뛰어 4큐비트에서 보드 DSP를 99.3% 태워 버립니다. 제안 설계는 **31-32개로 불박이**입니다. 3.3절 “곱셈은 클럭당 1개”의 직접적 결과이고, 이 논문의 설계 철학이 숫자 하나로 증명되는 지점입니다.

둘째, LUT가 진짜 병목입니다. 기존 방식은 3큐비트에서 이미 66.9%, 4큐비트에서 **626%** (즉 보드 용량의 6배)를 요구해 합성 자체가 실패합니다. 제안 설계는 3큐비트에서 29,486개로 **78% 감축**(136,437 → 29,486)을 달성하고, 9큐비트에서 63.9%로 버팁니다. 다만 10큐비트에서 113%로 넘어가 실패합니다.

셋째, 가속기만의 비용을 뽑아 보면 스케일링이 선명합니다. 참조값(24,853 LUT / 24,445 FF)을 빼면 QMA 자체의 비용은 다음과 같습니다 (아래는 필자가 계산한 파생값입니다).

n	$N = 2^n$	QMA LUT	QMA FF
2	4	약 4,100	약 2,600
4	16	약 5,700	약 4,200
6	64	약 11,500	약 10,400
8	256	약 54,400	약 36,300
9	512	약 105,300	약 70,500

n	$N = 2^n$	QMA LUT	QMA FF
10	1024	약 205,300	약 138,700

n 이 $8 \rightarrow 9 \rightarrow 10$ 으로 갈 때 QMA LUT가 대략 **2배씩** 됩니다 — 정확히 $O(N) = O(2^n)$ 입니다. 이론과 실측이 맞아떨어지죠. 작은 n 에서 비율이 안 맞는 건 CORDIC·곱셈기 같은 **고정 비용**이 지배하기 때문입니다.

넷째, 왜 하필 9큐비트에서 멈추는가. $O(N)$ 이라도 $N = 2^n$ 은 지수함수라 결국 벽에 부딪힙니다. 큐비트 하나 늘 때마다 LUT/FF가 2배씩 필요하니, 이 최적화는 **벽을 없앤 게 아니라 3큐비트에서 9큐비트로 6칸 밀어낸** 것입니다. 그리고 그 자원을 먹는 주체는 4.1절에서 말한 대로 **BRAM이 아닌 FF에 올린 상태벡터와 N 개 병렬 덧셈기**입니다.

5.3 자원 사용량 — Artix-7 (원문 Table III, p.7)

엔트리급 보드에서도 통하는지 보려고 **ORCA 코어** + Xilinx **Artix-7 XC7A100T** 조합으로 같은 실험을 했습니다.

n	LUT (사용률)	FF (사용률)	DSP (사용률)
	(Max: 63,400)	(Max: 126,800)	(Max: 240)
2	17,053 (26.9%)	19,785 (15.6%)	31 (12.9%)
3	17,625 (27.8%)	20,402 (16.1%)	31 (12.9%)
4	18,748 (29.6%)	21,395 (16.9%)	31 (12.9%)
6	25,554 (40.3%)	27,806 (27.8%)*	32 (13.3%)
8	51,168 (80.7%)	53,545 (42.2%)	32 (13.3%)
9 (실패)	84,739 (133.7%)	87,692 (69.2%)	32 (13.3%)

참조값: QC 가속기가 없는 에뮬레이터의 LUT/FF/DSP = **12,555 / 17,154 / 4**

원문 표기 그대로 옮겼습니다. 다만 $n = 4$ 는 원문에 18.748 로 인쇄되어 있는데 18,748 의 오타이고, * 표시한 $n = 6$ 의 FF 사용률 27.8%는 $27,806/126,800 = 21.9\%$ 이므로 원문 계산이 틀린 것으로 보입니다 (나머지 행은 전부 검산이 맞습니다).

결론은 **제안 설계는 8큐비트까지 성공, 기존 설계는 같은 보드에서 2큐비트가 한계**입니다. 자원이 심하게 제약된 환경에서도 훨씬 복잡한 Weighted-MaxCut 문제를 다룰 수 있다는 뜻이죠. DSP는 여기서도 31-32개로 고정이고, Artix-7의 DSP가 240개뿐인데도 13%밖에 안 씁니다 — **DSP는 이제 병목이 전혀 아니고, LUT가 유일한 벽**이라는 게 표 전체의 메시지입니다.

6. 결론

원문 p.8, §VI

저자들이 스스로 정리한 기여는 네 가지입니다. (1) Weighted-MaxCut에 초점을 맞춰 QAOA 연산을 하드웨어 효율적으로 재구성했다. (2) 임베디드 시스템용 파이프라인 기반 아키텍처를 설계해 **QMA**를 만들었다. (3) QAOA 에뮬레이터를 빠르게 짚어내는 도구 **QEX**를 만들었다. (4) 제안-베이스라인 양쪽을 FPGA에 프로토타이핑해 성능-에너지-자원을 종합 비교했다.

논문이 명시적으로 다루지 않은 한계도 함께 알아 두면 좋습니다. 고정소수점 정밀도가 결과 품질에 미치는 영향, 어떤 고전 옵티마이저를 썼고 몇 번 반복해야 수렴하는지, QMA 내부의 비트 폭, 그리고 실제 Weighted-MaxCut 문제에서 근사비(approximation ratio)가 얼마나 나오는지 — 이 넷은 이 논문의 범위 밖입니다. 즉 이 논문은 “정확도” 논문이 아니라 “이 계산을 얼마나 작고 빠르게 태울 수 있나” 논문입니다.

7. 이 프로젝트에 어떻게 쓰이나

여러분은 QAOA가 아니라 **Grover** 에뮬레이터를 만듭니다. 그럼에도 이 논문이 읽을 값어치가 있는 이유와, 무엇을 가져오고 무엇을 버려야 하는지 구체적으로 정리합니다.

7.1 왜 이 논문인가 — 형제 논문 관계

이 논문의 참고문헌 [10] 이 “S. Choi and W. Lee, Developing a Grover’s quantum algorithm emulator on standalone FPGAs: optimization and implementation, AIMS Mathematics, vol. 9, no. 11, pp. 30939-30971, 2024” 입니다. 이걸 여러분 폴더의 ../papers/2. Grover 논문 (LPSoC)/Developing a Grover's quantum algorithm emulator on standalone FPGAs.pdf 와 **같은 논문**이고, 저자 Seonghyun Choi와 Woojoo Lee가 이 QAOA 논문의 제1저자-교신저자입니다. 두 편은 같은 연구실이 같은 SoC 골격(RVX/RISC-V + AXI IP + standalone FPGA) 위에 각각 Grover와 QAOA를 올린 **자매 논문**입니다. 그래서 Fig. 4의 아키텍처는 여러분이 만들 것의 청사진에 가깝습니다. (참고문헌 [9] El-Araby et al. 도 여러분 폴더의 3. QCE-FPGA 논문/Towards_Complete_and_Scalable_Emulation...pdf 입니다 — 그쪽은 “HPC에 붙는 가속기” 진영의 대표 논문으로, 이 논문이 대비시키는 상대편입니다.)

7.2 그대로 가져올 것

(1) SoC 골격 — Fig. 4를 그대로 베끼세요. RISC-V 코어 + AXI 인터커넥트 + 가속기 IP + SRAM + APB 주변장치. 여러분의 project1/hardware/grover_ip.v 는 지금 clk/rst_n/start/done/result 만 있는 순수 RTL 모듈인데, 이걸 **AXI-Lite 슬레이브로 감싸는** 것이 다음 단계입니다. 이 논문의 API 4종을 Grover로 번역하면 이렇게 됩니다.

QAOA (원문)	Grover 대응 (제안)	대응 레지스터
set_cost_hamiltonian	load_values / set_oracle	grover_min_ip.v 의 value[0..N-1]
set_parameter	set_threshold / set_iters	T_INIT, R_ITERS
activate_maxcut	activate_grover	start 펄스
get_expectation	get_result	result, result_val

(2) $U_A = H^{\otimes n} D$ 라는 기본 연산 — Grover가 정확히 이 형태입니다. 이게 가장 중요한 이론적 전이입니다. Grover의 확산 연산자를 보세요.

$$U_\psi = 2|s\rangle\langle s| - I = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$$

가운데 $2|0\rangle\langle 0| - I = \text{diag}(1, -1, -1, \dots, -1)$ 는 **대각행렬**입니다. 그리고 오라클 U_f 도 정답 인덱스에만 -1 을 곱하는 **대각행렬**이고요. 즉 **Grover 한 바퀴 = 대각 × (아다마르 대각 아다마르)** 로, 원문 Eq. 14의 $(H^{\otimes n} D_M)(H^{\otimes n} D_C)$ 와 구조가 똑같습니다. 이 논문의 파이프라인은 Grover에 그대로 태울 수 있습니다.

(3) H_1 스케일링과 비트 시프트 (원문 Eq. 15-16). $1/\sqrt{2^n}$ 을 매 원소에 곱하지 말고 밖으로 빼서 2^n 로 모은 뒤 시프트로 처리하는 기법. 고정소수점 설계에서 정규화 상수를 다루는 표준 수법이니 익혀 두세요.

(4) “곱셈은 직렬, 덧셈은 병렬” 원칙. Table II의 DSP 31-32 고정이 이 원칙의 증거입니다. 여러분의 grover_ip.v 는 지금 S_DIFF 에서 for (i=0;i<N;i=i+1) amp[i] <= two_mean - amp[i]; 로 N 개 뺄셈을 한 클럭에 병렬 수행하는데, 이건 이미 이 원칙과 같은 방향입니다. **N이 커져도 DSP를 안 늘리는 게 목표**라는 것만 기억하세요.

(5) 자원 스케일링 감각 — 여러분의 상한선을 미리 알 수 있습니다. 상태벡터를 FF에 올리는 한 큐비트당 LUT/FF가 2배씩 됩니다. 이 논문이 Kintex-7에서 9큐비트, Artix-7에서 8큐비트가 한계였습니다. Grover는 (7.3에서 설명할) 복소수가 필요 없어 저장-연산이 절반이므로 같은 보드에서 대략 **1큐비트 정도 더** 갈 수 있다고 기대할 만합니다. 목표 큐비트 수를 정할 때 이 표를 근거로 쓰세요.

(6) QEX식 파라미터화. 큐비트 수를 파라미터로 받아 레지스터 배열과 연산 유닛을 자동 구성한다는 발상. 여러분 코드는 이미 parameter integer N = 4, NBITS = 2 로 이 방향을 잡고 있습니다. 다만 INIT_AMP 를 손으로 넣고 있는데(18'sd32768), N에서 자동으로 유도되게 만드는 게 QEX의 정신입니다.

(7) 소프트웨어 오버헤드가 지배한다는 경고. 4.3절 말미에서 봤듯 9큐비트에서도 가속기 시간(약 $85\mu s$)보다 나머지(약 $250\mu s$)가 큼니다. 여러분의 grover_tb.v 는 100MHz 클럭으로 RTL만 재고 있어 이 오버헤드가 안 보입니다. SoC에 통합할 때 반드시 다시 측정하세요.

7.3 가져오면 안 되는 것

(1) 비용/믹서 해밀토니안, γ/β , 변분 최적화 — 전부 불필요합니다. Grover는 연산자가 고정이고 튜닝할 파라미터가 없습니다. 반복 횟수도 $\approx \frac{\pi}{4}\sqrt{N}$ 로 미리 계산되죠 (여러분 grover_tb.v 의 R_ITEERS(1) 주석 그대로). 원문 §II 전체와 §III-B의 D_C 유도는 Grover에선 통째로 증발하고, D 는 그냥 상수 ± 1 테이블입니다.

(2) CORDIC과 복소수 곱셈 — 표준 Grover에는 필요 없습니다. 이게 가장 큰 절약 포인트입니다. 여러분 grover_ip.v 주석에 이미 정확히 적혀 있죠:

“핵심 단순화: 표준 Grover는 진폭이 '실수'로만 유지된다 (시작이 실수 균등중첩, 오라클은 ± 1 , 확산도 실수 선형). → 복소수가 필요 없어 실수 고정소수점 하나면 충분.”

맞습니다. QAOA는 D 의 원소가 $e^{i\theta}$ 라 CORDIC(16단) + 복소수 곱셈(실수 곱 4개)이 필수지만, Grover는 D 의 원소가 ± 1 이라 **스테이지 1~4가 통째로 부호 반전 하나로 줄어듭니다**. 파이프라인이 5단에서 사실상 1단이 되고, DSP 31개가 0개에 가까워집니다.

(3) 원문 Eq. 16의 $H_1 D_M H_1$ 경로를 Grover 확산에 그대로 쓰지 마세요 — 더 나은 길이 있습니다. Grover의 확산은 평균 중심 반사로 $a_i \mapsto 2\bar{a} - a_i$ 인데, 이건 합산 트리 + 시프트 + 뺄셈으로 곱셈 0개, 아다마르 통과 0번에 끝납니다. 논문 방식대로 $H_1 D H_1$ 로 가면 아다마르를 두 번 통과해야 하니 $2(N+1)$ 클럭이 듭니다. 여러분의 현재 grover_ip.v 가 이미 더 나은 방식($S_SUM \rightarrow \text{mean} = \text{sum} \ggg \text{NBITS} \rightarrow S_DIFF \rightarrow \text{two_mean} - \text{amp}[i]$)을 쓰고 있습니다. 이걸 논문을 안 따라간 게 아니라 Grover의 특수성을 제대로 활용한 것이니 바꾸지 마세요.

(4) 기댓값 읽기 → Grover는 argmax 또는 Born 샘플링. get_expectation 대신 여러분 코드의 S_MEAS처럼 |amp| 최대 인덱스를 고르거나, grover_min_ip.v 의 LFSR 방식처럼 확률적으로 샘플링합니다.

7.4 그런데 — 이 논문이 진짜로 필요해지는 순간 두 가지

7.3에서 “CORDIC은 필요 없다”고 했지만, 여러분 폴더를 보면 곧 필요해집니다.

(1) 일반화 Grover. ../papers/4. Grover 응용 논문/Generalized Grover's Algorithm for Multiple Phase Inversion States.pdf 는 오라클이 -1 이 아니라 임의 위상 $e^{i\phi}$ 를 새기는 변형입니다. 그 순간 진폭이 복소수가 되고, D 가 $\text{diag}(e^{i\phi_1}, \dots)$ 가 되며 — 이 논문의 5단 파이프라인(CALCULATE_RAD → NORMALIZE_RAD → CORDIC → 1_MULT → N_ADD)이 거의 그대로 필요해집니다. $\gamma_k \hat{H}_C(l, l)$ 자리에 ϕ_l 을 꽂으면 끝입니다. 즉 이 논문은 일반화 Grover 하드웨어의 완성된 설계도입니다.

(2) 고전 루프가 있는 Grover — grover_min_ip.v 가 바로 그것. Grover 자체엔 변분 루프가 없지만, 여러분이 구현한 Dürr-Høyer 최솟값 탐색(DHA)에는 외부 고전 루프가 있습니다: Grover 라운드 → 후보 측정 → 고전 검증 → 기준 T 갱신 → 반복. 이걸 QAOA의 “실행 → 기댓값 → 파라미터 갱신 → 반복”과 구조가 동일합니다.

그래서 4.1절의 HW/SW 분할이 그대로 적용됩니다. 지금 grover_min_ip.v 는 고전 검증·갱신(S_UPD : if (value[cand] < T) begin T <= value[cand]; best <= cand; end)과 라운드 카운팅(round, ROUNDS)을 FSM 하드웨어 안에서 하고 있습니다. 이 논문의 철학을 따른다면 그 부분은 RISC-V 소프트웨어로 올리고, IP는 “ T 를 받아 Grover 라운드를 돌고 후보 하나를 뽑는” 순수 함수로 남기는 게 맞습니다. 이유는 세 가지입니다.

- **자원:** 비교기, 갱신 로직, 라운드 카운터가 LUT를 먹는데, 5.2절에서 봤듯 LUT가 유일한 병목입니다. 그 예산을 상태벡터 확장에 쓰는 게 낫습니다.
- **유연성:** 탐색 예산(ROUNDS)이나 종료 조건을 바꾸려고 재합성하는 건 낭비입니다. 소프트웨어면 그냥 다시 컴파일하면 됩니다.
- **선례:** 이 논문이 정확히 그렇게 했습니다. 옵티마이저를 하드웨어에 넣지 않고 Rocket 코어에 남겼죠.

정리하면 — 양자 부분(상태벡터 전개)은 HW, 고전 부분(판정·갱신·루프 제어)은 SW. 이 한 줄이 이 논문에서 여러분이 가져갈 가장 큰 설계 원칙입니다.

7.5 다음 단계 제안

1. grover_ip.v 를 AXI-Lite 슬레이브로 래핑하고 7.2절 표의 API 4종을 레지스터로 노출 → 검증: RISC-V C 코드에서 activate_grover 호출 후 result 를 읽어 기존 grover_tb.v 와 같은 답(인덱스 2)이 나오는지 확인.
2. N, NBITS, INIT_AMP 를 NUM_QUBIT 하나에서 자동 유도되게 파라미터화 (QEX 방식) → 검증: $n = 2 \dots 8$ 로 바꿔 가며 합성이 통과하고 tb가 PASS하는지 확인.

3. Table II 방식으로 **큐비트별 LUT/FF/DSP 표**를 뽑아 이 논문 표와 나란히 비교 → 검증: Grover가 복소수를 안 쓰므로 같은 n 에서 이 논문보다 자원이 적게 나와야 정상. 안 그러면 어딘가 잘못된 것.
4. grover_min_ip.v 의 S_UPD / ROUNDS 를 **소프트웨어로 이관** → 검증: 최솟값 인덱스 3, 값 1이 그대로 나오는지 확인.
5. 고정소수점 비트 폭은 이 논문이 답을 안 주니 **Precision-aware fixed-point emulation of Grover's algorithm** 을 읽고 결정.

부록: 원문 수식 색인

수식	내용	원문 위치
Eq. 1	비용·믹서 앤사츠 연산자 정의	p.2
Eq. 2	p 레이어 앤사츠 상태	p.2
Eq. 3	기댓값 F_p (양자적 = 확률적)	p.2
Eq. 4	Weighted-MaxCut 비용함수	p.2
Eq. 5	비용·믹서 해밀토니안	p.3
Eq. 6	$Z_i Z_j$ 대각원소 (독해 주의)	p.3
Eq. 7-8	유니터리 게이트 형태	p.3
Eq. 9	$R_X(2\beta_k)$ 행렬	p.3
Eq. 10-11	고윳값 $\lambda_1 = e^{-i\beta_k}, \lambda_2 = e^{i\beta_k}$	p.3
Eq. 12	$R_X = H\Lambda_M H$ — 분해의 출발점	p.3
Eq. 13	$U_M = H^{\otimes n} D_M H^{\otimes n}$	p.3
Eq. 14	$U_M U_C = (H^{\otimes n} D_M)(H^{\otimes n} D_C) - O(N)$ 달성	p.3
Eq. 15	수정 아다마르 H_1 정의	p.4
Eq. 16	$\frac{1}{2^n}(H_1 D_M)(H_1 D_C)$ — 곱셈 제거	p.4
Eq. 17-19	$D_C(l, l) = e^{-i\gamma_k \hat{H}_C(l, l)}$	p.4
Eq. 20	$D_M(l, l) = e^{iu(n, l)\beta_k}, u = 2HW(l-1) - n - \text{popcount}$ 로 계산	p.4

굵게 표시한 다섯 개가 이 논문의 논리 사슬입니다. 나머지는 그 사슬을 잇는 유도 과정입니다.